

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Рубежный контроль №1

«Связи и запросы вариант 18 Б»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-32Б _____ (Подругин Г.Р.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Задание и модель данных

Рубежный контроль №1 по дисциплине «Парадигмы и конструкции языков программирования». Вариант запросов Б, предметная область 18: «Музыкальное произведение — Оркестр». Выполнил Подругин Григорий Романович, группа ИУ5-32Б, зачётная книжка 25У1243.

Цель работы — реализовать обработку связанных коллекций на Python: соединение по идентификаторам, сортировку, подсчёт и отбор данных. В программе представлены отношения один ко многим и многие ко многим.

Запросы варианта Б

Б1. Вывести все связанные музыкальные произведения и их основные оркестры по отношению 1:N. Упорядочить результат по названию произведения по возрастанию.

Б2. Вывести каждый оркестр и количество закреплённых за ним произведений по отношению 1:N. Упорядочить по количеству по возрастанию; при равенстве — по названию оркестра. Пустой оркестр получает количество 0.

Б3. Вывести произведения, названия которых заканчиваются на «ов», и списки исполняющих их оркестров по отношению M:N. Например, условию соответствуют «Вальс цветов» и «Танец эльфов».

При переносе условий примера «сотрудник» заменён музыкальным произведением, «отдел» — оркестром, «фамилия» — названием произведения. Окончание «ов» сохранено. Для сравнения строк без учёта регистра применяется casefold().

Классы и отношения

MusicalWork: id, title, duration_minutes, orchestra_id. Длительность в минутах — условный количественный признак. В Б2 требуется число записей, поэтому длительности не суммируются. Orchestra: id и name. WorkOrchestra: work_id и orchestra_id.

В отношении 1:N поле orchestra_id задаёт один основной оркестр произведения. В отношении M:N отдельный список links допускает исполнение одного произведения несколькими оркестрами и нескольких произведений одним оркестром. Все соединения и подсчёты используют ID.

Тестовые данные

В памяти создаются 4 произведения, 3 оркестра и 5 записей связи. Каждая коллекция содержит от 3 до 5 объектов. Репертуар и длительности заданы как учебные данные.

Оркестры

ID	Название
1	Симфонический оркестр
2	Камерный оркестр
3	Филармонический оркестр

Музыкальные произведения

ID	Название	Минуты	ID оркестра
1	Вальс цветов	8	1
2	Танец эльфов	3	2
3	Танец рыцарей	5	1
4	Полет шмеля	2	3

Связи многие ко многим

ID произведения	ID оркестра
1	1
1	2
2	2
3	1
4	3

«Вальс цветов» закреплён за Симфоническим оркестром в 1:N, а в M:N связан также с Камерным оркестром. Поэтому Б2 даёт Камерному оркестру 1 произведение, а Б3 включает его в список оркестров «Вальса цветов». Это различие демонстрирует независимость отношений.

Результаты выполнения

Запуск из корня репозитория на Python 3.10 или новее. Внешние библиотеки для программы и проверок РК1 не требуются.

`python3 rk_01/main.py`

```
$ python3 rk_01/main.py
РК-1. Вариант предметной области 18. Запросы Б
Б1. Произведения и основные оркестры, по названию произведения
('Вальс цветов', 'Симфонический оркестр')
('Полет шмеля', 'Филармонический оркестр')
('Танец рыцарей', 'Симфонический оркестр')
('Танец эльфов', 'Камерный оркестр')

Б2. Оркестры и количество произведений, по возрастанию
('Камерный оркестр', 1)
('Филармонический оркестр', 1)
('Симфонический оркестр', 2)

Б3. Произведения с окончанием 'ов' и их оркестры, связь M:N
Вальс цветов: Симфонический оркестр, Камерный оркестр
Танец эльфов: Камерный оркестр
```

Рисунок 1. Фактический вывод программы, оформленный по сохранённому stdout.

Проверка результатов

Б1: 4 пары упорядочены по названию произведения. Б2: Камерный — 1, Филармонический — 1, Симфонический — 2; сумма равна 4. Б3: «Вальс цветов» — два оркестра, «Танец эльфов» — один. Названия без окончания «ов» исключены.

`python3 -m unittest rk_01.test_queries -v`
Все 6 тестов пройдены. Проверены точные результаты Б1–Б3, пустые коллекции, пустой оркестр, одинаковые названия при разных ID, регистр и отсутствие связей. Полный журнал: rk_01/results/checks.txt.

Источник задания

Страница курса COURSE_PCPL_MAIN_2026 и PDF задания РК1, страницы 1, 2 и 4. Сверено 28 октября 2026 года. В программе применены list comprehensions, генераторы, sorted(), sum(), any() и lambda-функции. Все три запроса выполнены.

https://github.com/ugapanyuk/courses_current/blob/main/python/rk/pcpl/pcpl_rk_1.pdf

Текст программы

```
"""РК-1: предметная область 18, запросы Б. Python 3.10+."""


class MusicalWork:
    """Произведение: ID, название, длительность и основной оркестр."""

    def __init__(self, id, title, duration_minutes, orchestra_id):
        self.id = id
        self.title = title
        self.duration_minutes = duration_minutes
        self.orchestra_id = orchestra_id


class Orchestra:
    """Оркестр: ID и название."""

    def __init__(self, id, name):
        self.id = id
        self.name = name


class WorkOrchestra:
    """Связь M:N: ID произведения и ID исполняющего его оркестра."""

    def __init__(self, work_id, orchestra_id):
        self.work_id = work_id
        self.orchestra_id = orchestra_id


def sample_data():
    """Учебное распределение репертуара, а не концертная афиша."""
    orchestras = [
        Orchestra(1, "Симфонический оркестр"),
        Orchestra(2, "Камерный оркестр"),
        Orchestra(3, "Филармонический оркестр"),
    ]
    works = [
        MusicalWork(1, "Вальс цветов", 8, 1),
        MusicalWork(2, "Танец эльфов", 3, 2),
        MusicalWork(3, "Танец рыцарей", 5, 1),
        MusicalWork(4, "Полет шмеля", 2, 3),
    ]
    links = [
        WorkOrchestra(1, 1),
        WorkOrchestra(1, 2),
        WorkOrchestra(2, 2),
        WorkOrchestra(3, 1),
        WorkOrchestra(4, 3),
    ]
    return works, orchestras, links
```

Текст программы Продолжение

```
def query_b1(works, orchestras):
    """Б1: связь 1:N, сортировка по названию произведения."""
    one_to_many = [
        (work.title, orchestra.name)
        for work in works
        for orchestra in orchestras
        if work.orchestra_id == orchestra.id
    ]
    return sorted(one_to_many, key=lambda row: (row[0].casefold(), row[1]))


def query_b2(works, orchestras):
    """Б2: количество произведений по связи 1:N, по возрастанию."""
    counts = [
        (orchestra.name,
         sum(1 for work in works if work.orchestra_id == orchestra.id))
        for orchestra in orchestras
    ]
    # При равном количестве дополнительно сортируем по названию оркестра.
    return sorted(counts, key=lambda row: (row[1], row[0].casefold()))


def query_b3(works, orchestras, links):
    """Б3: названия на «ов» и оркестры через отдельную связь M:N."""
    return [
        (work.title, [
            orchestra.name
            for orchestra in orchestras
            if any(link.work_id == work.id
                   and link.orchestra_id == orchestra.id for link in links)
        ])
        for work in sorted(works, key=lambda work: work.title.casefold())
        if work.title.casefold().endswith("ов")
    ]


def main():
    works, orchestras, links = sample_data()
    print("РК-1. Вариант предметной области 18. Запросы Б")
    print("Б1. Произведения и основные оркестры, по названию произведения")
    for row in query_b1(works, orchestras):
        print(row)

    print("\nБ2. Оркестры и количество произведений, по возрастанию")
    for row in query_b2(works, orchestras):
        print(row)

    print("\nБ3. Произведения с окончанием 'ов' и их оркестры, связь M:N")
    for title, names in query_b3(works, orchestras, links):
        print(f"{title}: {', '.join(names) if names else 'нет оркестров'}")


if __name__ == "__main__":
    main()
```